

ConeShare-Guard：面向多输出逻辑锥共享的 结构感知逻辑优化方案

基于粗粒度结构选择、ABC 组合优化与 CEC 门控验证的
BLIF 网表优化方法

项目	内容
赛题	CPIPC 赛题十：多输出逻辑锥逻辑共享优化
作品名称	ConeShare-Guard
最终指标	37097 AIG nodes / 最大层级 20 / CEC 30/30 / 无效输出 0
队伍名称	七号特工

摘要

本方案面向赛题十“多输出逻辑锥逻辑共享优化”，作品名称为 ConeShare-Guard。方案以 ABC 逻辑综合工具为基础平台，围绕 BLIF 网表的结构特征、输出锥共享程度和候选优化安全性构建最终执行流程。正式提交阶段通过粗粒度结构特征选择器选择合适的 ABC 命令序列，并在部分具备共享潜力的结构分组上尝试受控的无关条件指导重替换后处理。所有最终输出必须经过 ABC 的组合等价性检查，并满足 AIG 节点数下降、逻辑层级不增加等指标约束，因此方案将优化效果和正确性验证统一到同一条可复现执行路径中。

公开 30 个 BLIF 用例上的最终结果为 AIG 节点数总计 37097，层级和为 275，最大层级为 20，CEC 30/30，无效最终输出数为 0，优化时间总计 113.026 s，峰值 RSS 47.273 MB。方案只使用 PI/PO 分组、.names/cube/fanin 分布、二输入门比例、高扇入 SOP 标志、近似二输入 AIG 标志、TFI-overlap、输出锥簇分组等粗粒度结构统计量，不使用文件名、目录名、hash、case ID 或公开样本精确指纹。

关键词：多输出逻辑锥；ABC；BLIF；AIG；TFI-overlap；CEC；结构感知选择器；逻辑共享优化

目录

摘要	2
目录	3
1 赛题理解与设计目标	5
2 总体方案架构	5
3 多输出逻辑锥共享建模	7
3.1 TFI-overlap 定义	7
3.2 输出锥簇共享潜力评分	7
4 结构特征提取与选择器设计	8
4.1 结构特征选择器伪代码	9
5 ABC 命令序列组合设计	9
6 受控重替换后处理设计	10
7 正确性验证与质量约束	11
7.1 最终输出接受准则	11
8 公开集实验结果与增益分析	13
9 运行时间、内存与复杂度控制	15
10 端口顺序敏感性与风险控制	15
10.1 端口顺序敏感性说明	16
11 可复现流程与提交包结构	16
12 结论	17
附录 A 结构特征选择规则摘要	18
附录 B ABC 命令序列配置摘要	18
附录 C 30 个公开用例逐例结果	18
附录 D 受控重替换后处理逐例明细	20

附录 E CEC 日志摘要与复现证据	22
参考资料	22

1 赛题理解与设计目标

赛题要求围绕多输出逻辑锥之间的共享优化开展算法设计。输入与输出均为 BLIF 网表，基础优化平台为开源逻辑优化工具 ABC。测试用例由公开 BLIF 用例和辅助验收 BLIF 用例构成，所有用例均需完成有效性校验后进入关键指标统计。

本方案采用“结构感知、轻量执行”的原则。每个 BLIF 用例先被转化为可解释的结构特征向量，再由选择器确定一条匹配的 ABC 命令序列，并在共享潜力较高的结构上执行小预算重替换后处理。这样既体现多输出逻辑锥共享优化的主题，又避免在单用例执行时承担全量组合搜索的时间和内存开销。

表 1-1 赛题约束与方案策略对应关系

项目	设计约束与目标	方案对应策略
功能正确性	优化后网表必须与输入逻辑等价	最终输出均执行 ABC CEC
节点数	AIG 节点数越小越优	以最终输出 AIG 节点数下降作为第一优化目标
层级数	AIG CO 最大层级越小越优	后处理结果仅在逻辑层级不增加时接受
运行时间	单线程计时，耗时越小越优	最终执行不跑全量命令序列组合搜索
峰值内存	执行过程峰值内存越小越优	优化驱动程序单进程串行调度 ABC 子进程
隐藏用例适应性	辅助用例同样计入最终结果	仅使用粗粒度结构统计，避免样本身份绑定

2 总体方案架构

ConeShare-Guard 的总体流程由三层组成：结构分析层、优化执行层和验证选择层。结构分析层从 BLIF/AIG 中抽取分组特征；优化执行层根据结构特征选择器输出选择 ABC 命令序列，并在特定共享结构分组上尝试受控重替换后处理；验证选择层通过组合等价性检查和节点/层级约束确认最终输出。

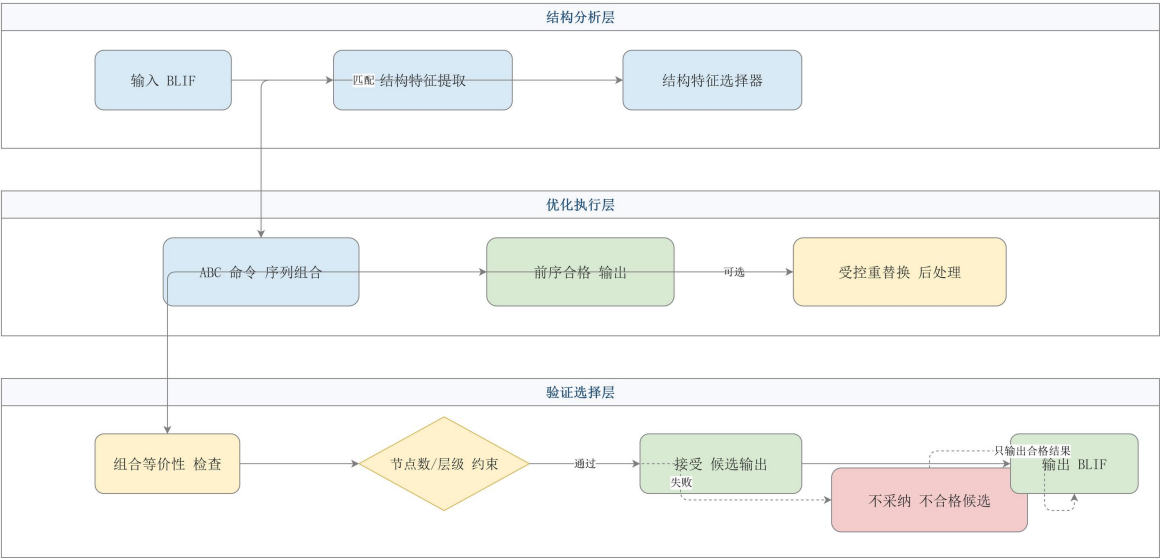


图 2-1 ConeShare-Guard 总体算法架构

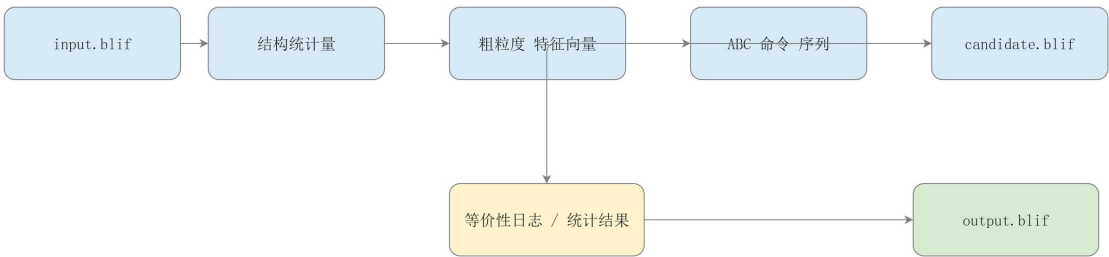


图 2-2 数据对象流：从 BLIF 到候选输出与验证日志

表 2-1 总体架构三层职责

层次	输入	输出	作用
结构分析层	BLIF 网表、strash 后结构统计	粗粒度特征向量 F(N)	判断网表规模、SOP/AIG-like 程度与多输出共享潜力
优化执行层	特征向量与选择规则	候选 BLIF 输出	选择并执行 dc2、sop/fx、FRAIG、DSD/DCH、受控重替换等路径
验证选择层	候选输出、CEC 结果、AIG 统计指标	最终输出 BLIF	由组合等价性检查和指标约束保证等价、节点收益与层级约束

架构中的核心边界是：结构特征选择器负责给出结构上合理的候选路径，最终输出必须由组合等价性检查和指标约束确认。该边界使得方案能够在保持正确性的前提下，稳定吸收不同优化路径带来的节点收益。

3 多输出逻辑锥共享建模

单输出局部窗口优化容易忽略不同输出之间的公共逻辑。对于输出集合 $O=\{o_1,o_2,...,o_m\}$ ，每个输出 o_i 的 $TFI(o_i)$ 表示能够到达该输出的内部节点集合。若两个输出的 TFI 集合存在较大交集，则存在潜在共享机会。

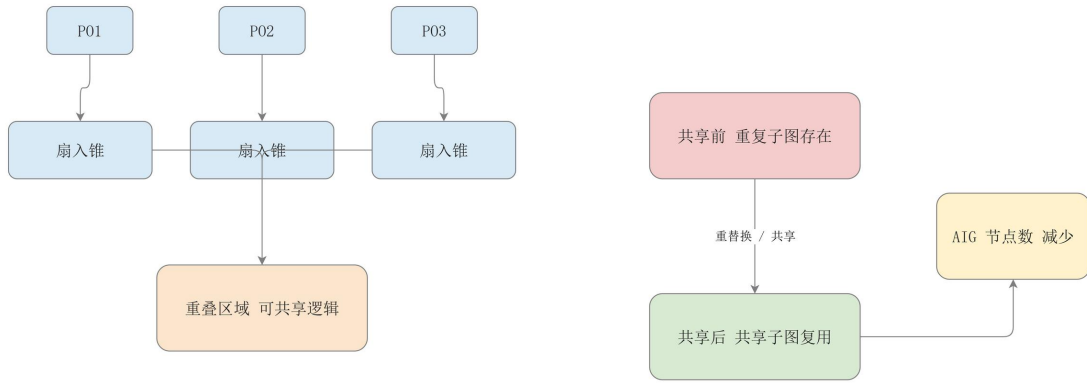


图 3-1 多输出 TFI-overlap / cone sharing 示意图

3.1 TFI-overlap 定义

设 $|TFI(o_i)|$ 表示输出 o_i 的扇入锥节点数，两个输出之间的共享比例可用式（3-1）刻画。该比例越高，说明两个输出锥存在越多可复用内部结构。

$$\text{overlap}(o_i, o_j) = |TFI(o_i) \cap TFI(o_j)| / \min(|TFI(o_i)|, |TFI(o_j)|) \quad (3-1)$$

当 $\text{overlap}(o_i, o_j)$ 高于阈值时，该输出对被计入高重叠输出对。对所有输出对进行统计后，可得到共享内部节点数、高重叠输出对数量、最大输出锥簇规模等统计量，用于识别后续共享增强路径。

3.2 输出锥簇共享潜力评分

为避免使用精确用例指纹，受控后处理只使用粗粒度共享统计量构造结构分组。概念上，输出锥簇共享潜力评分可由式（3-2）表达：

$$\text{Score} = \alpha \cdot \text{norm}(\text{shared internal nodes}) + \beta \cdot \text{norm}(\text{high-overlap output pairs}) + \gamma \cdot \text{norm}(\text{max shared-cone group size}) \quad (3-2)$$

该分数不直接作为精确匹配键，而是用于形成“无明显共享结构簇、低共享结构簇、中等共享结构簇、高共享结构簇”等稳定分组。结构特征选择器和后处理仅基于分组结果做粗粒度决策。

4 结构特征提取与选择器设计

结构特征选择器的目标是根据 BLIF/AIG 的结构特征选择最合适的优化路径。特征提取不依赖文件身份信息，而是将网络规模、逻辑表达形态和多输出共享程度转化为可解释的结构向量。

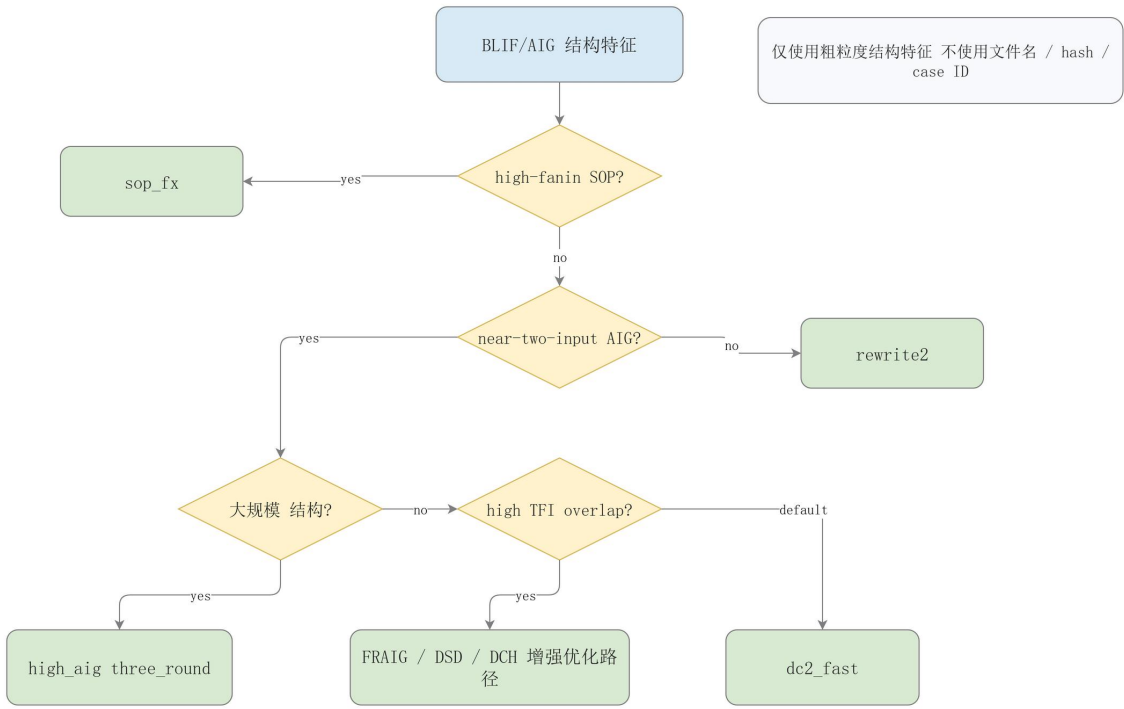


图 4-1 粗粒度结构特征选择器决策树

表 4-1 结构特征定义与用途

特征类别	代表特征	作用
规模特征	PI/PO 分组、.names 分组、规模等级、运行规模分组	区分小规模、中规模和大规模网络，控制优化强度
SOP/AIG 形态	fanin 分布、cube 分组、二输入门比例、高扇入 SOP、近似二输入 AIG	判断先进行 SOP/fx 抽取，还是直接进入 AIG 优化
共享特征	TFI-overlap、共享内部节点数、TFI 重复比例、输出锥簇分组	识别多输出共享潜力和受控后处理适用范围
质量约束	程序返回状态、CEC 结果、节点数/层级变化	确认最终输出的等价性和指标优势

4.1 结构特征选择器伪代码

Algorithm 1: Coarse Structural Feature Selector

Input : BLIF network N
Output: ABC command sequence P

1. $F \leftarrow \text{ExtractCoarseFeatures}(N)$
2. if $F.\text{high_fanin_sop}$ and $F.\text{scale_grade} \in \{\text{tiny}, \text{small}\}$:
 return sop_fx
3. if $F.\text{near_two_input_aig}$ and $F.\text{runtime_size_bin} = \text{large}$:
 return $\text{high_aig_three_round}$
4. if $F.\text{r7b_eligible}$ or $F.\text{high_tfi_overlap}$:
 return $\text{r7b_r7win_fraig_high}$
5. if $F.\text{dsd_dch_eligible}$:
 return $\text{r25_dsd_dch_cleanup}$
6. if $F.\text{scale_grade} = \text{medium}$ and not $F.\text{near_two_input_aig}$:
 return rewrite2
7. return dc2_fast

5 ABC 命令序列组合设计

ABC 命令序列组合的作用不是盲目堆叠命令，而是根据结构类型选择成本与收益匹配的优化路径。默认路径保持低成本；高扇入 SOP 结构先使用 SOP/fx 抽取共享；高 TFI-overlap 或 DSD/DCH 友好结构使用更有针对性的清理与重写。

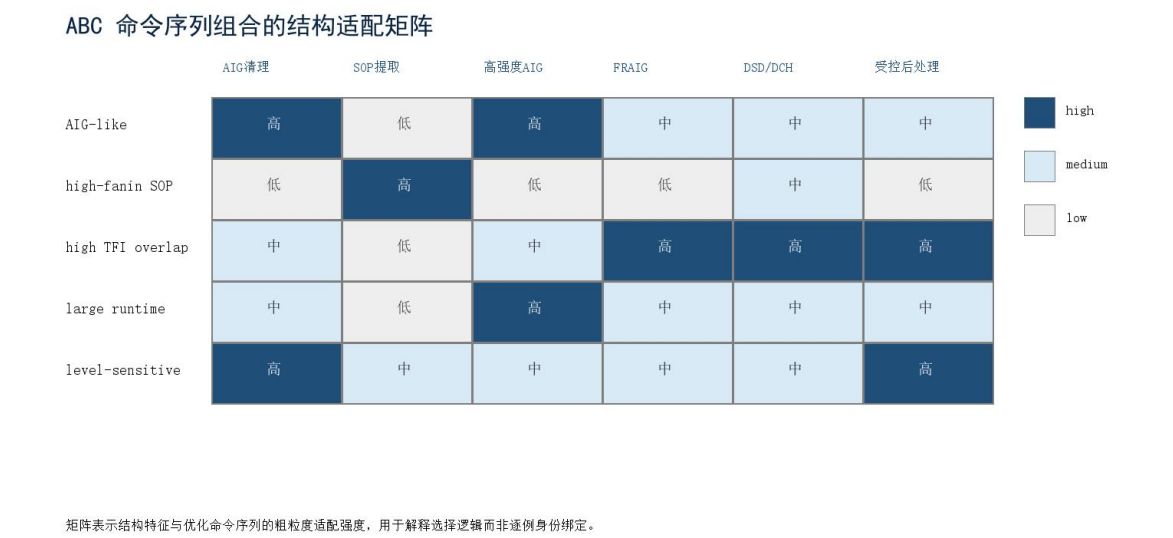


图 5-1 ABC 命令序列组合的结构适配矩阵

表 5-1 ABC 命令序列组合设计摘要

结构类型	优化路径	主要 ABC 命令	设计意图
默认 AIG-like	AIG 清理与重写	strash; balance; rewrite -z; refactor -z; dc2; balance	低成本通用 AIG 优化

结构类型	优化路径	主要 ABC 命令	设计意图
中等非 AIG-like	两轮重写	两轮 rewrite/refactor/balance	轻量重复重写
高扇入 SOP	SOP 共享提取	sop; fx; strash; balance; rewrite -z; refactor -z; balance	先提取 SOP/fx 共享
大规模 near-AIG	高强度 AIG 重写	多轮 rewrite/refactor/resub/dc2/balance	对大规模结构投入更高优化强度
高 TFI-overlap	FRAIG 增强优化	窗口化优化 + FRAIG + 高强度清理	利用共享窗口和 FRAIG
DSD/DCH 友好结构	DSD/DCH 分解清理	DSD + DCH + cleanup	利用分解和 DCH 清理
高共享潜力结构	受控重替换后处理	resub -K 8 -N 1 -M 1 -F 1 + cleanup	共享潜力结构上的受控重替换

6 受控重替换后处理设计

受控重替换后处理是面向高共享潜力结构分组的局部优化步骤。它不是无边界的 transduction 引擎，而是小预算、近似无关条件指导的重替换单元，目标是在保持逻辑层级不回归的前提下进一步降低 AIG 节点数。

```
strash; resub -K 8 -N 1 -M 1 -F 1; strash; dc2; rewrite -z; balance
```

表 6-1 受控重替换参数与设计目的

参数/步骤	解释	设计目的
K=8	限制 cut size	控制搜索空间
N=1	限制新增节点	避免面积反弹
M=1	限制替换复杂度	控制运行时间
F=1	允许更积极局部替换	捕捉共享收益
dc2/rewrite/balance	清理并控制层级	防止局部优化带来结构恶化

后处理结果只有在组合等价性检查通过、节点数下降、层级不增加时才进入最终输出。该设计使局部重替换带来的节点收益能够被稳定吸收，同时避免用层级或正确性风险换取面积收益。

Algorithm 2: Bounded Resubstitution Post-processing

Input : quality-checked output B

Output: final output O

1. Extract coarse TFI-overlap and shared-cone features from B
2. if structural group is not suitable:
return B
3. $C \leftarrow$ run bounded resubstitution command sequence on B
4. if process failed, timed out, or bad return code:
return B
5. if not CEC(original, C):
return B
6. if nodes(C) < nodes(B) and level(C) <= level(B):

```
return C
7. return B
```

7 正确性验证与质量约束

功能等价是该赛题的前置有效性条件。本方案将组合等价性检查作为最终输出的硬约束，所有进入最终结果的 BLIF 均需通过 ABC CEC，并同时满足节点数、层级和运行状态约束。

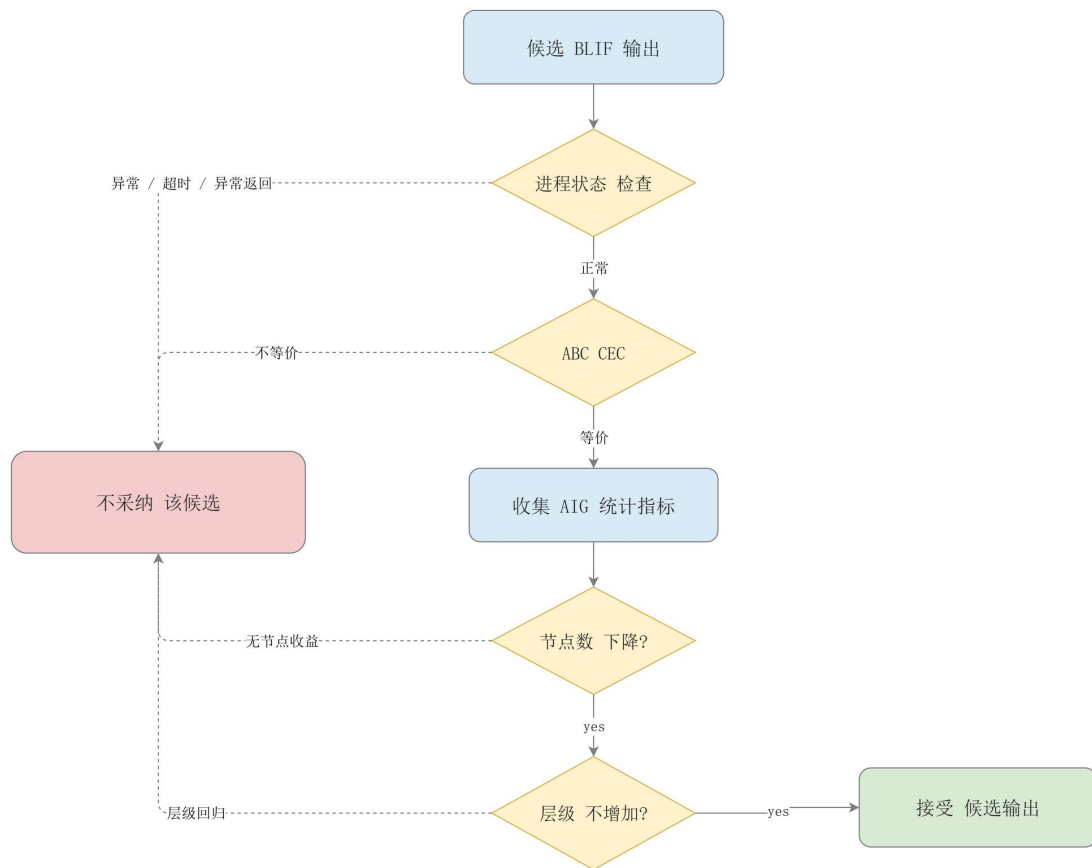


图 7-1 候选输出质量约束状态机

7.1 最终输出接受准则

Algorithm 3: Candidate Acceptance Criterion

Input : original BLIF I, candidate BLIF C, current best valid output B
Output: final BLIF O

1. if candidate optimization crashed or timed out:
 discard C and keep B
2. if $ABC_CEC(I, C) \neq \text{equivalent}$:
 discard C and keep B
3. $stat_C \leftarrow \text{CollectAIGStats}(C)$
 $stat_B \leftarrow \text{CollectAIGStats}(B)$

```
4. if stat_C.nodes < stat_B.nodes and stat_C.level <= stat_B.level:
    return C
5. return B
```

该准则的重点不是描述失败路径，而是保证最终输出同时具备三项性质：逻辑等价、节点数收益和层级不回归。因此，最终报告主要呈现通过验证后的结果和优势，不把中间失败候选作为方案主线。

表 7-1 最终输出质量约束

质量约束	进入最终输出的条件	体现的方案优势
程序状态	优化过程正常结束，无异常返回	执行路径稳定、可复现
组合等价性	ABC CEC 判断等价	保证功能正确性
AIG 节点数	候选输出节点数低于当前有效输出	面积指标持续下降
逻辑层级	最大层级不增加	避免用时序深度换面积
输出来源	最终输出来自通过验证的优化结果	结果可解释、可复核

8 公开集实验结果与增益分析

最终公开 30 例结果来自真实运行日志和提交包复现记录。汇总指标显示，本方案在 CEC 30/30、无效最终输出 0 的条件下得到 AIG 节点数 37097，层级和 275，最大层级 20。

表 8-1 运行环境与实验约束

项目	内容
CPU	AMD Ryzen 7 7840HS with Radeon 780M Graphics
内存	约 29.7 GB 物理内存
OS	Microsoft Windows 11 家庭版 中文版, 64-bit, 10.0.26200
Python	3.12.13
ABC build	/MT Release Windows build
单线程约束	优化驱动程序单进程/单线程调度；每个优化、CEC、统计步骤调用一个 ABC 进程
超时设置	优化超时 300s、CEC 超时 300s、统计超时 120s
内存测量	运行日志中记录 peak RSS

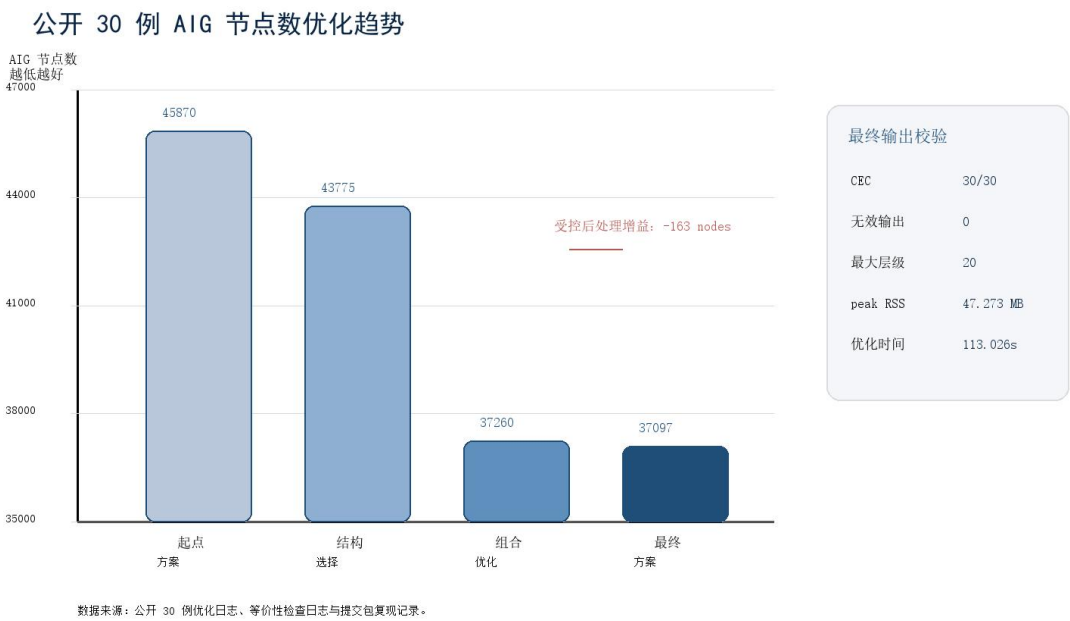


图 8-1 公开 30 例 AIG 节点数优化趋势

受控重替换后处理的接受 / 拒绝 / 跳过分布



图 8-2 受控重替换后处理接受/拒绝/跳过分布

从阶段性路径看，方案先由结构特征选择带来大幅节点下降，再通过 ABC 命令序列组合压低整体节点数，最后由受控重替换后处理进一步带来 163 个节点收益，层级和下降 2，最大层级不变。剔除最佳单例后仍有 103 个节点收益，剔除前两例后仍有 70 个节点收益，说明最终优势不是由单个公开用例单独支撑。

表 8-2 公开集汇总指标

指标	结果	说明
AIG 节点数	37097	公开 30 例最终输出节点数总计
层级和 / 最大层级	275 / 20	层级受控，无高层级换节点
组合等价性检查	30/30	公开集全部等价
无效输出 / 异常返回	0 / 0	未产生无效最终输出、无入口异常
优化 / 等价性检查 / 统计时间	113.026s / 83.700s / 16.204s	公开 30 例累计
峰值 RSS	47.273 MB	峰值常驻内存

表 8-3 公开用例逐例结果摘录（前 10 例；完整表见附录 C）

用例	前序节点	最终节点	节点收益	层级	优化时间/s	CEC	优化路径	异常输出
tc_public_1	85	85	0	8	3.663	通过	r1l_gia_deepsyn_tiny@both	0
tc_public_2	19	19	0	4	0.456	通过	sop_fx@clean	0
tc_public_3	94	94	0	8	3.990	通过	r1l_gia_deepsyn_tiny@clean	0

用例	前序节点	最终节点	节点收益	层级	优化时间/s	CEC	优化路径	异常输出
tc_public_4	61	61	0	8	2.319	通过	r11_gia_deepsyn_tiny@inputs	0
tc_public_5	65	65	0	2	0.462	通过	dc2_fast@clean	0
tc_public_6	162	162	0	4	0.429	通过	sop_fx@clean	0
tc_public_7	191	191	0	3	1.562	通过	r11_gia_deepsyn_tiny@clean	0
tc_public_8	193	193	0	3	0.449	通过	sop_fx@clean	0
tc_public_9	194	194	0	4	2.063	通过	r11_gia_deepsyn_tiny@clean	0
tc_public_10	101	101	0	4	1.626	通过	r11_gia_deepsyn_tiny@clean	0

9 运行时间、内存与复杂度控制

正式路径以“单用例轻量选择”为基本原则。结构特征提取复杂度与网表节点和边规模近似线性相关；ABC 优化耗时由被选中的命令序列决定，而不是所有命令序列的组合成本。受控重替换后处理通过共享结构分组触发，并通过 K、N、M 参数限制候选枚举规模。

表 9-1 运行时间与内存汇总

时间/内存项	数值	解释
累计优化时间	113.026s	公开 30 例累计优化时间
累计 CEC 时间	83.700s	公开 30 例累计等价验证时间
累计统计时间	16.204s	公开 30 例累计统计时间
峰值 RSS	47.273 MB	正式评测路径峰值内存
后处理时间增量	+7.985s	受控重替换后处理带来的公开集优化时间增量

公开集峰值 RSS 为 47.273 MB，主要来自较大用例的 ABC 优化过程。优化驱动程序不并行启动多个 ABC 实例，避免并发导致的内存峰值叠加。

10 端口顺序敏感性与风险控制

隐藏用例适应性的关键在于避免样本身份绑定。本方案将所有决策限定在可解释的粗粒度结构特征上，并明确排除文件名、目录名、hash 和公开样本精确指纹。

表 10-1 特征使用边界

禁止项	原因	当前处理
文件名 / 目录名 / case ID	会形成公开集身份绑定	不作为结构特征选择器输入
hash / exact line count	可能构成精确指纹	不使用
精确 PI/PO/name/cube 组合	可能只匹配公开集个例	仅使用 bins 和结构分布
端口名 / 输出顺序指纹	隐藏集可能变化	禁止用于选择条件
只依赖公开集最优解的离线搜索	无法在隐藏提交阶段复现	不进入正式提交路径
未稳定的自定义 ABC 命令	存在广泛退化风险	不作为最终方案默认路径

10.1 端口顺序敏感性说明

已知风险是端口顺序扰动下存在数值敏感性：AIG 节点数为 37222，最大层级为 20，CEC 30/30，无效最终输出 0，优化路径/节点数/层级不一致数量为 14/15/6。该风险反映不同端口顺序下局部优化路径可能发生变化，但不属于等价性失败。正式复现路径以提交包内记录的选择规则、命令序列和输出为准。

该结果说明最终方案的正确性不依赖端口顺序假设；即使局部优化路径随端口顺序变化，最终输出仍由 CEC 与节点/层级约束确认，从而保证可复现的有效结果。

11 可复现流程与提交包结构

提交包保留单用例入口、配置文件、ABC 可执行文件、公开集输出结果、CEC 复现脚本和日志。单用例接口将待优化 BLIF 放置为 input.blif，执行 optimize_one.py 后生成 output.blif。

```
# 从仓库根目录验证公开集 CEC
python tools/verify_all_cec.py --abc submit/bin/abc.exe --cases data/tc_public --
outputs submit/results/final_public --log submit/logs/reproduce_cec.log

# 从 submit/ 内部验证
python tools/verify_all_cec.py --abc bin/abc.exe --cases ../data/tc_public --outputs
results/final_public --log logs/reproduce_cec.log

# 单个隐藏/验收 BLIF 入口
python tools/optimize_one.py --abc bin/abc.exe --input input.blif --output output.blif
--selector configs/final_selector.yaml --pipelines configs/pipelines.yaml
```

表 11-1 提交包结构说明

提交包内路径	作用
--------	----

提交包内路径	作用
bin/abc.exe	正式 ABC 可执行文件，/MT Release build
configs/final_selector.yaml	粗粒度结构特征选择规则
configs/pipelines.yaml	ABC 命令序列组合与受控后处理配置
tools/optimize_one.py	单例优化入口
tools/verify_all_cec.py	公开集/包内 CEC 复现脚本
results/final_public/	正式公开集输出
logs/reproduce_cec.log	包内复现 CEC 日志
docs/algorithm_design.docx	算法设计方案文档
reproduce.md	提交包复现说明

12 结论

ConeShare-Guard 将多输出共享感知、粗粒度结构特征选择、ABC 命令序列组合、受控重替换后处理与组合等价性检查组合成统一的正式执行流程。方案的最终优势体现在：节点数显著下降、最大层级保持较低、公开 30 例全部 CEC 等价、运行时间和内存开销仍处于轻量范围。

最终结果在公开 30 例上达到 CEC 30/30、无效最终输出 0、AIG 节点数 37097、层级和 275、最大层级 20。阶段性对比表明，结构选择、ABC 命令序列组合和受控重替换分别贡献了不同层面的收益，最终方案在结果强度、正确性和可复现性之间取得了较好的平衡。

附录 A 结构特征选择规则摘要

正式提交配置中的结构特征选择规则仅使用粗粒度结构统计量。完整机器可执行配置保存在 `configs/final_selector.yaml`；该文件用于复现，不在正文中作为算法命名反复引用。

表 A-1 结构特征选择规则摘要

规则类别	使用的专业特征	作用
规模分组	PI/PO 数量区间、.names 数量区间、运行规模分组	控制优化命令序列的搜索强度
逻辑形态	fanin 分布、cube 分布、二输入门比例、高扇入 SOP、近似二输入 AIG	判断 SOP 抽取、AIG 重写或分解清理路径
共享结构	TFI-overlap、共享内部节点数、输出锥簇规模	识别多输出共享潜力
安全约束	程序返回状态、CEC 结果、节点数变化、层级变化	决定候选是否可进入最终输出

附录 B ABC 命令序列配置摘要

完整 ABC 命令序列配置保存在 `configs/pipelines.yaml`。正文只保留专业层面的配置摘要，避免把内部实验名称误写成算法概念。

表 B-1 ABC 命令序列配置摘要

配置类别	主要 ABC 命令	用途
通用 AIG 优化	strash, balance, rewrite, refactor, dc2	低成本节点数优化与层级整理
SOP 结构优化	sop, fx, strash	在高扇入 SOP 结构上提取共享逻辑
FRAIG 增强优化	fraig 相关命令与清理命令	利用功能归约识别等价结构
DSD/DCH 分解清理	DSD/DCH 与 cleanup	处理分解友好结构并消除冗余
受控重替换后处理	resub -K 8 -N 1 -M 1 -F 1	在共享潜力结构上进行小预算重替换

附录 C 30 个公开用例逐例结果

表 C-1 公开 30 例逐例结果

用例	前序节点	最终节点	节点收益	层级	优化时间/s	CEC	优化路径	异常输出
tc_public_1	85	85	0	8	3.663	通过	r11_gia_deepsyn_tiny@both	0
tc_public_2	19	19	0	4	0.456	通过	sop_fx@clean	0

用例	前序节点	最终节点	节点收益	层级	优化时间/s	CEC	优化路径	异常输出
tc_public_3	94	94	0	8	3.990	通过	r11_gia_deepsyn_tiny@clean	0
tc_public_4	61	61	0	8	2.319	通过	r11_gia_deepsyn_tiny@inputs	0
tc_public_5	65	65	0	2	0.462	通过	dc2_fast@clean	0
tc_public_6	162	162	0	4	0.429	通过	sop_fx@clean	0
tc_public_7	191	191	0	3	1.562	通过	r11_gia_deepsyn_tiny@clean	0
tc_public_8	193	193	0	3	0.449	通过	sop_fx@clean	0
tc_public_9	194	194	0	4	2.063	通过	r11_gia_deepsyn_tiny@clean	0
tc_public_10	101	101	0	4	1.626	通过	r11_gia_deepsyn_tiny@clean	0
tc_public_11	328	328	0	7	0.677	通过	r10_medium_fraig_cleanup@clean	0
tc_public_12	1367	1366	1	14	1.552	通过	r10_medium_fraig_cleanup@clean+r29_mfs_strash_dc2_rwz_bal+r30b_odc_resub_fl	0
tc_public_13	1043	1043	0	13	1.857	通过	r7b_r7win_fraig_high@clean	0
tc_public_14	2044	2044	0	14	5.624	通过	r7b_r7win_fraig_high@outputs+r28_r27_mfs_fraig_dc2_bal+r29_mfs_strash_dc2_rwz_bal	0
tc_public_15	904	900	4	11	2.109	通过	r7b_r7win_fraig_high@clean+r28_r27_mfs_fraig_dc2_bal+r29_mfs_strash_dc2_rwz_bal+r30b_odc_resub_fl	0
tc_public_16	45	45	0	6	0.413	通过	dc2_fast@clean	0
tc_public_17	93	93	0	6	0.385	通过	dc2_fast@clean	0
tc_public_18	483	483	0	12	1.106	通过	r10_medium_fraig_cleanup@both+r29_mfs_strash_dc2_rwz_bal	0
tc_public_19	5157	5124	33	18	10.364	通过	r9_dc2_fraig_cleanup@inputs+r29_mfs_strash_dc2_rwz_b	0

用例	前序节点	最终节点	节点收益	层级	优化时间/s	CEC	优化路径	异常输出
							al+r30b_odc_resub_fl	
tc_public_20	5412	5382	30	20	11.553	通过	r9_dc2_fraig_cleanup@inputs+r29_mfs_strash_dc2_rwz_bal+r30b_odc_resub_fl	0
tc_public_21	5412	5400	12	18	11.700	通过	r9_dc2_fraig_cleanup@inputs+r29_mfs_strash_dc2_rwz_bal+r30b_odc_resub_fl	0
tc_public_22	10683	10623	60	20	20.484	通过	r9_dc2_fraig_cleanup@both+r29_mfs_strash_dc2_rwz_bal+r30b_odc_resub_fl	0
tc_public_23	1311	1311	0	13	22.560	通过	r11_gia_deepsyn_tiny@inputs+r29_mfs_strash_dc2_rwz_bal	0
tc_public_24	215	215	0	8	0.409	通过	dc2_fast@clean	0
tc_public_25	26	26	0	4	0.404	通过	dc2_fast@clean	0
tc_public_26	449	449	0	10	0.895	通过	r25_dsd_dch_cleanup@inputs+r28_r27_mfs_fraig_dc2_bal	0
tc_public_27	376	376	0	7	0.423	通过	dc2_fast@clean	0
tc_public_28	258	258	0	8	0.441	通过	r25_dsd_dch_cleanup@clean	0
tc_public_29	97	97	0	6	1.799	通过	r11_gia_deepsyn_tiny@inputs	0
tc_public_30	392	369	23	12	1.251	通过	r25_dsd_dch_cleanup@inputs+r28_r27_mfs_fraig_dc2_bal+r30b_odc_resub_fl	0

附录 D 受控重替换后处理逐例明细

表 D-1 受控后处理接受/拒绝/跳过明细

用例	处理结果	判定原因	节点变化	层级变化	共享结构分组
----	------	------	------	------	--------

用例	处理结果	判定原因	节点变化	层级变化	共享结构分组
tc_public_1	跳过	共享结构分组未命中	0	0	中等共享结构簇
tc_public_2	跳过	共享结构分组未命中	0	0	无明显共享结构簇
tc_public_3	跳过	共享结构分组未命中	0	0	低共享结构簇
tc_public_4	跳过	共享结构分组未命中	0	0	中等共享结构簇
tc_public_5	跳过	共享结构分组未命中	0	0	无明显共享结构簇
tc_public_6	跳过	共享结构分组未命中	0	0	无明显共享结构簇
tc_public_7	跳过	共享结构分组未命中	0	0	无明显共享结构簇
tc_public_8	跳过	共享结构分组未命中	0	0	无明显共享结构簇
tc_public_9	跳过	共享结构分组未命中	0	0	无明显共享结构簇
tc_public_10	跳过	共享结构分组未命中	0	0	无明显共享结构簇
tc_public_11	跳过	共享结构分组未命中	0	0	无明显共享结构簇
tc_public_12	接受	满足等价、节点与层级约束	-1	0	高共享结构簇
tc_public_13	拒绝	无节点收益	0	0	高共享结构簇
tc_public_14	跳过	共享结构分组未命中	0	0	高共享结构簇
tc_public_15	接受	满足等价、节点与层级约束	-4	0	高共享结构簇
tc_public_16	跳过	共享结构分组未命中	0	0	无明显共享结构簇
tc_public_17	跳过	共享结构分组未命中	0	0	无明显共享结构簇
tc_public_18	跳过	共享结构分组未命中	0	0	无明显共享结构簇
tc_public_19	接受	满足等价、节点与层级约束	-33	0	无明显共享结构簇
tc_public_20	接受	满足等价、节点与层级约束	-30	0	无明显共享结构簇
tc_public_21	接受	满足等价、节点与层级约束	-12	-2	无明显共享结构簇
tc_public_22	接受	满足等价、节点与层级约束	-60	0	无明显共享结构簇
tc_public_23	跳过	共享结构分组未命中	0	0	无明显共享结构簇

用例	处理结果	判定原因	节点变化	层级变化	共享结构分组
tc_public_24	跳过	共享结构分组未命中	0	0	高共享结构簇
tc_public_25	跳过	共享结构分组未命中	0	0	无明显共享结构簇
tc_public_26	跳过	共享结构分组未命中	0	0	高共享结构簇
tc_public_27	跳过	共享结构分组未命中	0	0	高共享结构簇
tc_public_28	跳过	共享结构分组未命中	0	0	高共享结构簇
tc_public_29	跳过	共享结构分组未命中	0	0	无明显共享结构簇
tc_public_30	接受	满足等价、节点与层级约束	-23	0	中等共享结构簇

附录 E CEC 日志摘要与复现证据

表 E-1 CEC 与包复现摘要

验证项	结果	相对路径
final CEC	30/30	logs/final_cec.log
package root CEC	30/30	submit/logs/reproduce_cec.log
package inside CEC	30/30	submit/logs/reproduce_cec_inside.log
zip-extracted root CEC	30/30	logs/r30b_formal_zip_extract_cec.log
zip-extracted inside CEC	30/30	logs/r30b_formal_zip_extract_inside_cec.log
hidden-style single-case smoke	equivalent	logs/r30b_formal_zip_hidden_smoke_tc_public_12_cec.log

参考资料

- [1] 中国研究生创新实践系列大赛管理平台，赛题十：多输出逻辑锥逻辑共享优化，
[https://cpipc.acge.org.cn/cw/contestNews/detail/10/2c9080189bd54d54019c07ea1f561497?](https://cpipc.acge.org.cn/cw/contestNews/detail/10/2c9080189bd54d54019c07ea1f561497?page=2)
 page=2
- [2] Berkeley ABC GitHub repository, ABC: System for Sequential Logic Synthesis and Formal Verification, <https://github.com/berkeley-abc/abc>
- [3] 最终提交材料清单与复现指标汇总，见提交包 reports 目录。
- [4] 包内复现说明与 CEC 日志，见 submit/reproduce.md 与 submit/logs 目录。
- [5] 提交包复现说明：submit/reproduce.md